

Order_ID	Order_Date	Region	Product_Category	Sales_Amount	Profit	
1001	1/5/2023	North	Electronics	25000	5000	
1002	1/10/2023	South	Furniture	18000	3000	
1003	2/15/2023	East	Clothing	12000	2500	
1004	2/18/2023	West	Electronics	30000	7000	
1005	3/2/2023	North	Clothing	15000	3500	
1006	3/12/2023	South	Electronics	22000	4500	
1007	3/20/2023	East	Furniture	27000	6000	
1008	4/5/2023	West	Clothing	13000	2800	
1009	4/18/2023	North	Furniture	32000	7500	
1010	5/1/2023	South	Electronics	29000	6800	Sales_data.csv

```
import pandas as pd

# Load a CSV file into a DataFrame
df = pd.read_csv('sales_data.csv')
print(df.head())

# Check for missing values in the DataFrame
df.isna().sum()

# convert a column to datetime format
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
print(df.dtypes)

# Calculate the total sales and total profit by region
region_summary = df.groupby('Region').agg({'Sales_Amount': 'sum', 'Profit': 'sum'})
print(region_summary)

# Average sales amount by product category
avg_sales = df.groupby('Product_Category')['Sales_Amount'].mean()
print(avg_sales)

# Top 5 orders by sales amount
top_orders = df.groupby('Order_ID')['Sales_Amount'].sum().nlargest(5)
print(top_orders)

# Summary report of sales and profit by region
region_summary.to_csv('region_summary_report.csv')
```

Region	Sales_Amount	Profit
East	39000	8500
North	72000	16000
South	69000	14300
West	43000	9800

region_summary_report.csv

Output

	Order_ID	Order_Date	Region	Product_Category	Sales_Amount	Profit
0	1001	2023-01-05	North	Electronics	25000	5000
1	1002	2023-01-10	South	Furniture	18000	3000
2	1003	2023-02-15	East	Clothing	12000	2500
3	1004	2023-02-18	West	Electronics	30000	7000
4	1005	2023-03-02	North	Clothing	15000	3500

Order_ID int64

Order_Date datetime64[ns]

Region object

Product_Category object

Sales_Amount int64

Profit int64

dtype: object

Region	Sales_Amount	Profit
--------	--------------	--------

East	39000	8500
------	-------	------

North	72000	16000
-------	-------	-------

South	69000	14300
-------	-------	-------

West	43000	9800
------	-------	------

Product_Category

Clothing	13333.333333
----------	--------------

Electronics	26500.000000
-------------	--------------

Furniture	25666.666667
-----------	--------------

Name: Sales_Amount, dtype: float64

Order_ID

1009	32000
------	-------

1004	30000
------	-------

1010	29000
------	-------

1007	27000
------	-------

1001	25000
------	-------

Name: Sales_Amount, dtype: int64

City1	City2	City3	City4	City5	
22	25	30	18	20	
23	26	31	19	21	
21	24	29	17	19	
24	27	32	20	22	
25	28	33	21	23	
26	29	34	22	24	
27	30	35	23	25	
28	31	36	24	26	
29	32	37	25	27	
30	33	38	26	28	
31	34	39	27	29	
32	35	40	28	30	
33	36	41	29	31	
34	37	42	30	32	
35	38	43	31	33	
36	39	44	32	34	
37	40	45	33	35	
38	41	46	34	36	
39	42	47	35	37	
40	43	48	36	38	
41	44	49	37	39	
42	45	50	38	40	
43	46	51	39	41	
44	47	52	40	42	
45	48	53	41	43	
46	49	54	42	44	
47	50	55	43	45	
48	51	56	44	46	
49	52	57	45	47	
50	53	58	46	48	temperature_data.csv

```
import numpy as np
import pandas as pd

# Load a CSV file into a NumPy array and display its shape

df = pd.read_csv('temperature_data.csv')
arr = df.shape
print("Shape of the array:", arr)

# Mean, median, and standard deviation of each City

mean_value = np.mean(df)
print("Mean:", mean_value)

med_value = np.median(df)
print("Median:", med_value)

std_dev = np.std(df)
print("Standard Deviation:", std_dev)

# Temperature data using min & max normalization
```

```

min_temp = np.min(df)
max_temp = np.max(df)

normalize_data = (df - min_temp) / (max_temp - min_temp)

print("Original: ", df)
print("Normalized Data: ", normalize_data)

# The city with the highest average temperature

max_temp_index = np.argmax(df.values, axis=0)

hottest_city = df.values[max_temp_index]
max_temp = df.values[max_temp_index].max()

print(f"The hottest city is {hottest_city} with {max_temp}°C.")

# Threshold with the city wise

threshold = 30
above_threshold = df[df > threshold]
print("Cities with temperature above threshold:\n", above_threshold)

```

Output

Shape of the array: (30, 5)

Mean: 36.5

Median: 36.5

C:\Users\MEDIA CELL\AppData\Roaming\Python\Python314\site-packages\numpy_core\fromnumeric.py:4026: FutureWarning: The behavior of DataFrame.std with axis=None is deprecated, in a future version this will reduce over both axes and return a scalar. To retain the old behavior, pass axis=0 (or do not pass axis)

```
return std(axis=axis, dtype=dtype, out=out, ddof=ddof, **kwargs)
```

Standard Deviation: City1 8.655441

City2 8.655441

City3 8.655441

City4 8.655441

City5 8.655441

dtype: float64

Original: City1 City2 City3 City4 City5

0 22 25 30 18 20

1 23 26 31 19 21

2 21 24 29 17 19

3	24	27	32	20	22
4	25	28	33	21	23
5	26	29	34	22	24
6	27	30	35	23	25
7	28	31	36	24	26
8	29	32	37	25	27
9	30	33	38	26	28
10	31	34	39	27	29
11	32	35	40	28	30
12	33	36	41	29	31
13	34	37	42	30	32
14	35	38	43	31	33
15	36	39	44	32	34
16	37	40	45	33	35
17	38	41	46	34	36
18	39	42	47	35	37
19	40	43	48	36	38
20	41	44	49	37	39
21	42	45	50	38	40
22	43	46	51	39	41
23	44	47	52	40	42
24	45	48	53	41	43
25	46	49	54	42	44
26	47	50	55	43	45
27	48	51	56	44	46
28	49	52	57	45	47
29	50	53	58	46	48

Normalized Data: City1 City2 City3 City4 City5

0	0.121951	0.195122	0.317073	0.024390	0.073171
1	0.146341	0.219512	0.341463	0.048780	0.097561
2	0.097561	0.170732	0.292683	0.000000	0.048780
3	0.170732	0.243902	0.365854	0.073171	0.121951
4	0.195122	0.268293	0.390244	0.097561	0.146341

5 0.219512 0.292683 0.414634 0.121951 0.170732
6 0.243902 0.317073 0.439024 0.146341 0.195122
7 0.268293 0.341463 0.463415 0.170732 0.219512
8 0.292683 0.365854 0.487805 0.195122 0.243902
9 0.317073 0.390244 0.512195 0.219512 0.268293
10 0.341463 0.414634 0.536585 0.243902 0.292683
11 0.365854 0.439024 0.560976 0.268293 0.317073
12 0.390244 0.463415 0.585366 0.292683 0.341463
13 0.414634 0.487805 0.609756 0.317073 0.365854
14 0.439024 0.512195 0.634146 0.341463 0.390244
15 0.463415 0.536585 0.658537 0.365854 0.414634
16 0.487805 0.560976 0.682927 0.390244 0.439024
17 0.512195 0.585366 0.707317 0.414634 0.463415
18 0.536585 0.609756 0.731707 0.439024 0.487805
19 0.560976 0.634146 0.756098 0.463415 0.512195
20 0.585366 0.658537 0.780488 0.487805 0.536585
21 0.609756 0.682927 0.804878 0.512195 0.560976
22 0.634146 0.707317 0.829268 0.536585 0.585366
23 0.658537 0.731707 0.853659 0.560976 0.609756
24 0.682927 0.756098 0.878049 0.585366 0.634146
25 0.707317 0.780488 0.902439 0.609756 0.658537
26 0.731707 0.804878 0.926829 0.634146 0.682927
27 0.756098 0.829268 0.951220 0.658537 0.707317
28 0.780488 0.853659 0.975610 0.682927 0.731707
29 0.804878 0.878049 1.000000 0.707317 0.756098

The hottest city is [[50 53 58 46 48]

[50 53 58 46 48]

[50 53 58 46 48]

[50 53 58 46 48]

[50 53 58 46 48]] with 58°C.

Cities with temperature above threshold:

City1 City2 City3 City4 City5

0 NaN NaN NaN NaN NaN

1	NaN	NaN	31.0	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	32.0	NaN	NaN
4	NaN	NaN	33.0	NaN	NaN
5	NaN	NaN	34.0	NaN	NaN
6	NaN	NaN	35.0	NaN	NaN
7	NaN	31.0	36.0	NaN	NaN
8	NaN	32.0	37.0	NaN	NaN
9	NaN	33.0	38.0	NaN	NaN
10	31.0	34.0	39.0	NaN	NaN
11	32.0	35.0	40.0	NaN	NaN
12	33.0	36.0	41.0	NaN	31.0
13	34.0	37.0	42.0	NaN	32.0
14	35.0	38.0	43.0	31.0	33.0
15	36.0	39.0	44.0	32.0	34.0
16	37.0	40.0	45.0	33.0	35.0
17	38.0	41.0	46.0	34.0	36.0
18	39.0	42.0	47.0	35.0	37.0
19	40.0	43.0	48.0	36.0	38.0
20	41.0	44.0	49.0	37.0	39.0
21	42.0	45.0	50.0	38.0	40.0
22	43.0	46.0	51.0	39.0	41.0
23	44.0	47.0	52.0	40.0	42.0
24	45.0	48.0	53.0	41.0	43.0
25	46.0	49.0	54.0	42.0	44.0
26	47.0	50.0	55.0	43.0	45.0
27	48.0	51.0	56.0	44.0	46.0
28	49.0	52.0	57.0	45.0	47.0
29	50.0	53.0	58.0	46.0	48.0

Month	Product_A_Revenue	Product_B_Revenue	Product_C_Revenue
Jan	50000	42000	38000
Feb	52000	45000	40000
Mar	55000	48000	42000

Apr	58000	50000	45000
May	60000	53000	47000
Jun	62000	55000	50000
Jul	65000	58000	52000
Aug	67000	60000	54000
Sep	69000	62000	56000
Oct	72000	65000	59000
Nov	75000	68000	62000
Dec	80000	72000	65000

monthly_revenue.csv

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Load Dataset

data = pd.read_csv('monthly_revenue.csv')

# line Chart Showing Monthly Revenue for Three Products

plt.figure(figsize=(12, 4))
plt.plot(data['Month'], data['Product_A_Revenue'], marker='o', label='Product A')
plt.plot(data['Month'], data['Product_B_Revenue'], marker='s', label='Product B')
plt.plot(data['Month'], data['Product_C_Revenue'], marker='^', label='Product C')
plt.title('Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.legend()
plt.grid(True)
plt.show()

# Bar Chart Showing Total Annual Revenue for Each Product

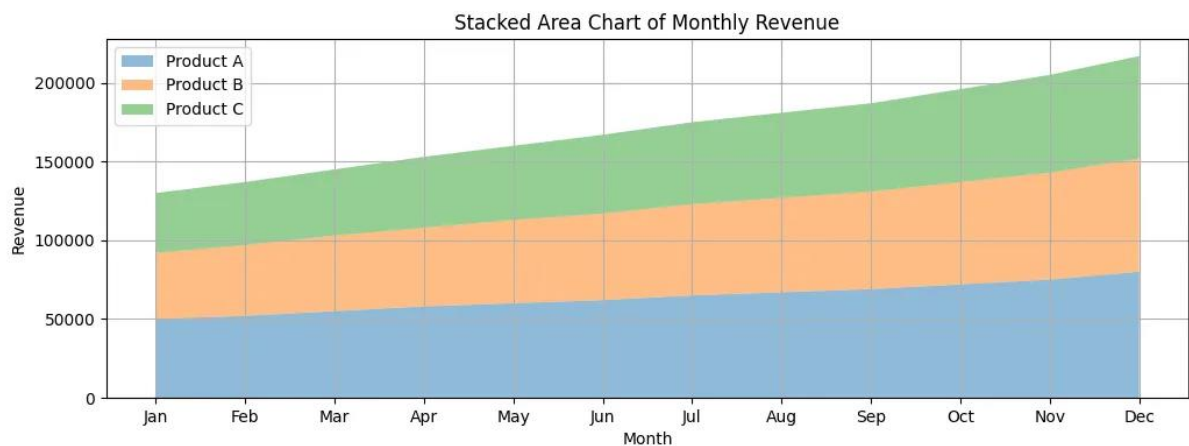
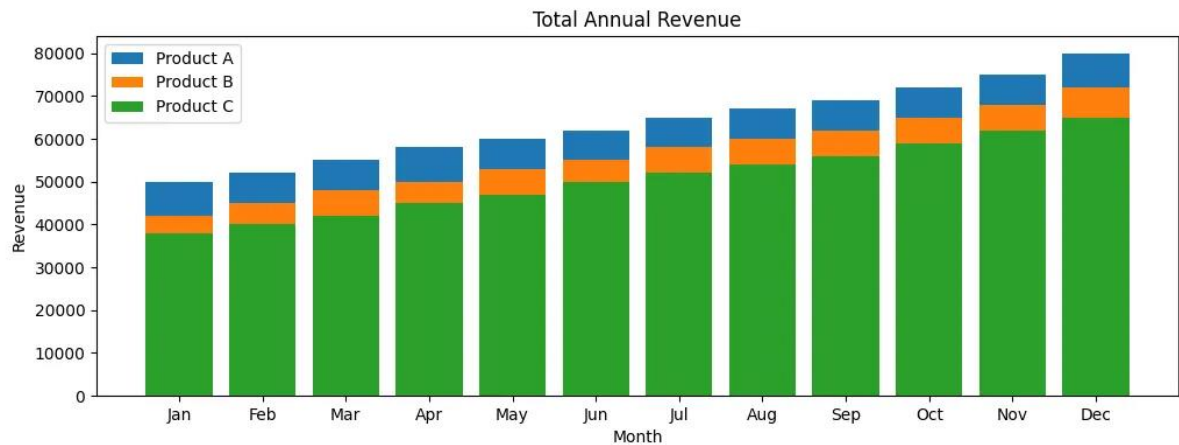
plt.figure(figsize=(12, 4))
plt.bar(data['Month'], data['Product_A_Revenue'], label='Product A')
plt.bar(data['Month'], data['Product_B_Revenue'], label='Product B')
plt.bar(data['Month'], data['Product_C_Revenue'], label='Product C')
plt.title('Total Annual Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.legend()
plt.show()

# stacked area chart Showing Monthly Revenue Distribution

plt.figure(figsize=(12, 4))
plt.stackplot(data['Month'],
              data['Product_A_Revenue'],
              data['Product_B_Revenue'],
              data['Product_C_Revenue'],
              labels=['Product A', 'Product B', 'Product C'], alpha=0.5)
```

```
plt.title('Stacked Area Chart of Monthly Revenue')
plt.xlabel('Month')
plt.ylabel('Revenue')
plt.legend(loc='upper left')
plt.grid(True)
plt.show()
```

Output



Employee_ID	Department	Experience_Years	Performance_Score	Salary
E101	IT	2	3.5	45000
E102	HR	5	4.2	55000
E103	Finance	7	4.6	65000
E104	IT	3	3.8	48000
E105	Marketing	6	4.1	60000
E106	HR	4	3.9	52000
E107	Finance	10	4.8	85000
E108	IT	8	4.5	78000
E109	Marketing	2	3.4	42000
E110	HR	9	4.7	80000

employee_performance.csv

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load Dataset

file_path = 'employee_performance.csv'
df = pd.read_csv(file_path)
print(df.head())

# Distribution of Salary

sns.histplot(df['Salary'], kde=True, color='red')
plt.title('Distribution of Salary')
plt.show()

# Salary distribution by Department

sns.boxplot(x='Department', y='Salary', data=df, palette='Set1', hue='Department',
legend=True)
plt.title('Salary Distribution by Department')
plt.show()

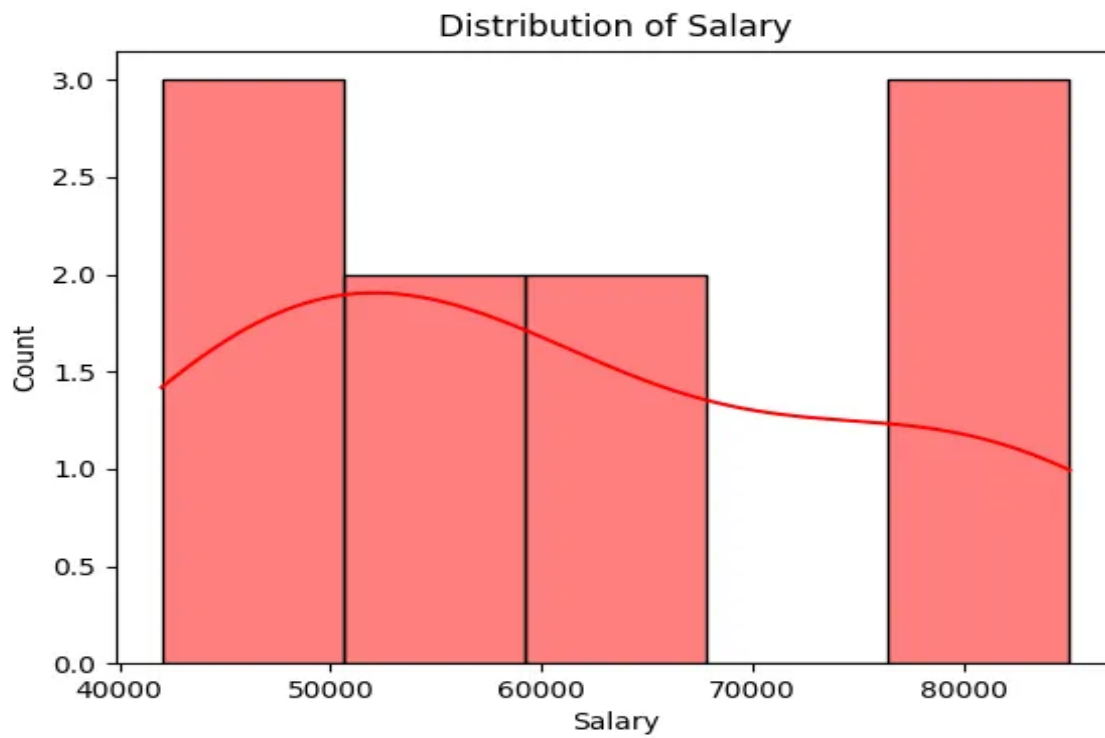
# Relationship between Experience and Performance Score

sns.scatterplot(x='Experience_Years', y='Performance_Score', data=df, hue='Department',
palette='Set2')
plt.title('Relationship between Experience and Performance Score')
plt.show()

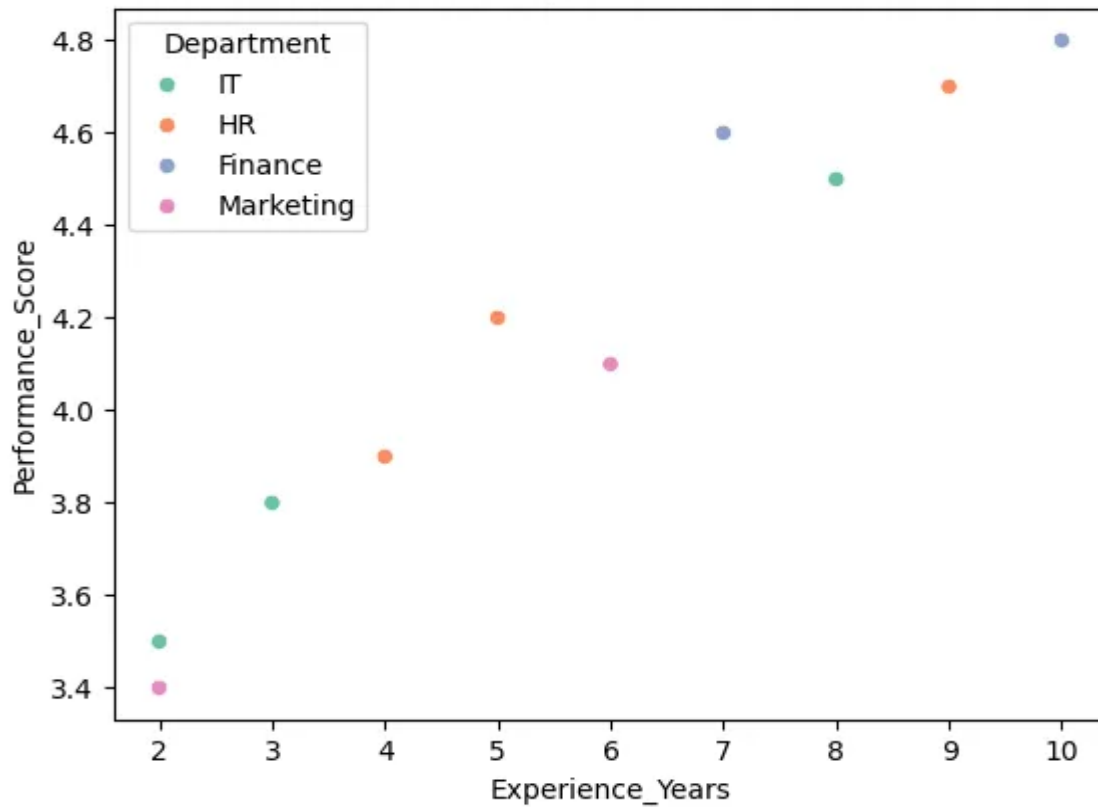
# Correlation Heatmap

corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

Output



Relationship between Experience and Performance Score



Correlation Heatmap

